IT5613
Socially Relevant Project Laboratory

Blockchain-based Decentralised Banking Application

A Project Report

submitted by

K Mohamed Jamal Basha	**2022506001**
Yuvaraj K	**2022506079**
Manobala P	**2022506081**

under the guidance of

C. Sunil Retmin Raj
Teaching Fellow, Department of Information Technology

Department of Information Technology
Madras Institute of Technology
Anna University, Chennai – 600 044

January – May 2025

Bonafide Certificate

Certified that this project, titled "Blockchain-based Decentralised Banking Application', is the Bonafide work of K MOHAMED JAMAL BASHA (2022506001), YUVARAJ K (2022506079), and MANOBALA P (2022506081), who carried out the project work under my supervision.

Dr. M.R. Sumalatha,
Head of the Department

Professor
Department of Information Technology
MIT Campus, Anna University
Chennai – 600 044

C. Sunil Retmin Raj
Supervisor

Teaching Fellow
Department of Information Technology
MIT Campus, Anna University
Chennai – 600 044

Acknowledgments

It is essential to mention the names of the people, whose guidance and encouragement made us accomplish this project.

We express our thankfulness to our project supervisor, C Sunil Retmin Raj, Professor, Department of Information Technology, MIT Campus, for providing invaluable support and assistance with encouragement which aided to complete this project.

We express our sincere gratitude to our Head of Department, Dr. M.R. Sumalatha, Professor, Department of Information Technology, MIT Campus, for providing the necessary motivation, focus, facilities, and resources required to work on the project to our fullest potential.

K Mohamed Jamal Basha 2022506001
Yuvaraj K 2022506079
Manobala P 2022506081

Table of Contents

Chapter	Title		Page No.
	Abstract		
	List of Figures		
	List of Abbreviations		
1	**Introduction**		**1**
	1.1	Overview	1
	1.2	Background	1
	1.3	Problem Statement	2
	1.4	Objective	2
	1.5	Scope	3
	1.6	Research Challenges	3
2	**Literature Survey**		**4**
	2.1	Overview of Blockchain-Based DAO	4
	2.2	Survey on Blockchain-Based Dapp	5
	2.3	Decentralised Banking Services	7
3	**Technology Stack**		**8**
	3.1	User Interface	8
	3.2	Backend	10
	3.3	Dependencies and Frameworks	11
4	**Workflow**		**15**
	4.1	Project Structure	15
	4.2	Architecture	16
	4.2	Flow of the Application	18
	4.3	Functionality	20
5	**Client-Side Application**		
	5.1	Local Application	
	5.2	Ganache Simulation	
	5.3	MetaMask Connection	
6	**Conclusion**		
	6.1	Potential Improvements	
	6.2	Conclusion	
7	**References**		

Abstract

Blockchain is essentially a decentralized database - a network and a database all in one. It is a peer-to-peer (P2P) network of computers called nodes, which all share data and code with each other.

Instead of a centralised database, all the transaction data in a Blockchain network are contained in bundles of records called blocks, which are chained together to create a public ledger that is secured and validated appropriately. Nodes on the network participate to ensure that all the copies of data distributed across the network are the same. If the database is modified, the information saved by all nodes will be noticed.

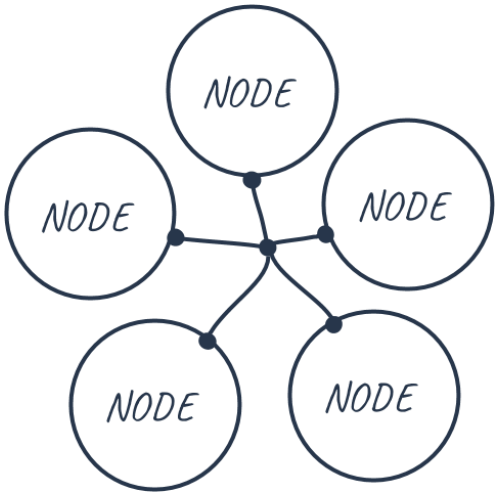


Fig 0.1: Decentralised System

Consider our present banking system - it has many different issues:

1. system users can modify bank details
2. system users can modify bank balance
3. transactions can potentially be intercepted and affected
4. users have to wait for a long time to deposit and withdraw money

There is security in our centralised banking system, but it is never really guaranteed. Banking data is stolen every single day and there is no feasible way to fix them once something has been affected.

A decentralised banking system, on the other hand, has no room for fraudulent activities, and it also causes no capital loss. Decentralized banking in blockchain is a huge leap in terms of describing the user's need for absolute non-transparency and confidentiality. Simply said, decentralized banking is a novel method of managing money that does not rely on established financial systems. Instead, it tracks funds and manages transactions through a decentralized network. Blockchain technology, which enables safe and open transactions, can be used to do this.

List of Figures

Figure No.	Title	Page No.
0.1	Decentralised System	1
2.1	Structure of a DApp	4
2.2	Architectures of DApps	5
4.1	Project Structure	15
4.2	Architecture Diagram	16
4.3	Flow Diagram	18
5.1	Truffle Contract Migration	24
5.2	Mocha Testing	24
5.3	Running Application	25
5.4	Home Page – No User Logged In	26
5.5	Dark Mode	26
5.6	Developers	27
5.7	After Connecting	27
5.8	Deposit	27
5.9	Ganache Simulated Accounts	28
5.10	Transaction History in Ganache	28
5.11	Transaction Request	29
5.12	Transaction Confirmation	30
5.13	Local Transaction History	30

List of Abbreviations

DApp	Decentralised Application
DAO	Decentralised Autonomous Organisation
ICO	Initial Coin Offering
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
NPM	Node Package Manager
DID	Decentralised Identity

1: Introduction

1.1: Overview

The rapid evolution of financial technologies has prompted a transformation in how banking services are delivered and accessed. Traditional centralized banking systems, while reliable, face limitations in transparency, trust, efficiency, and security. Blockchain technology presents a powerful alternative by enabling decentralized, transparent, and tamper-proof financial ecosystems. A blockchain-based decentralized banking application can empower users with direct control over their assets and transactions without the need for intermediaries, reducing operational costs and improving security and financial inclusion.

1.2: Background

Banking systems today rely heavily on centralized architectures, where banks and financial institutions control user data, validate transactions, and maintain ledgers. This centralization introduces single points of failure, susceptibility to fraud, and delays in transaction settlements. Blockchain, introduced with Bitcoin in 2009, is a distributed ledger technology that offers a way to manage transactions without a central authority. It ensures immutability, transparency, and consensus-driven validation.

With the rise of Decentralized Finance, blockchain is now being used to build decentralized applications or DApps that replicate and even enhance traditional financial services such as lending, payments, savings, and asset management. Leveraging smart contracts, these services operate autonomously and securely over blockchain networks, minimizing human intervention and institutional trust dependencies.

1.3: Problem Statement

Traditional banking systems are limited by centralization, which creates several issues:

- Lack of Transparency: Users have limited visibility into how their data and funds are managed.
- High Costs: Intermediaries and administrative overhead increase transaction costs.
- Security Risks: Centralized databases are prime targets for cyberattacks and data breaches.
- Financial Exclusion: Many individuals in underserved regions lack access to basic banking services.

There is a growing need for an alternative system that is secure, cost-efficient, transparent, and accessible. A blockchain-based decentralized banking application addresses these concerns by removing intermediaries and empowering users with full control over their financial activities.

1.4: Objective

The objective of this project is to:

- Design and develop a decentralized banking application using blockchain technology.
- Ensure the system supports core banking functionalities such as account creation, peer-to-peer transfers, and the like via smart contracts.
- Enhance security, transparency, and user autonomy in financial transactions.
- Demonstrate the advantages of decentralization over traditional banking in terms of efficiency, trust, and inclusion.
- Analyse performance, scalability, and compliance aspects of decentralized banking systems.

1.5: Scope

This report covers the following:

- Study and implementation of blockchain technology in the banking domain.
- Design of a decentralized architecture for banking operations using smart contracts.
- Implementation of basic financial services: deposits, withdrawals, transfers, and loans.
- Use of Ethereum or similar platforms for smart contract development.
- Evaluation of security, transaction costs, and user experience in the decentralized environment.
- Comparative analysis between centralized and decentralized banking models.

The scope does not include:

- Full regulatory compliance analysis.
- Real-world deployment with actual currencies (limited to test nets or token simulations).
- Integration with legacy banking infrastructure.

1.6: Research Challenges

- Scalability: Handling high transaction volumes efficiently on the blockchain.
- Security of Smart Contracts: Ensuring contracts are free from vulnerabilities and bugs.
- User Experience: Creating intuitive interfaces for users unfamiliar with blockchain.
- Regulatory Compliance: Navigating legal requirements and financial regulations.
- Interoperability: Ensuring seamless interaction with external systems and other blockchains.
- Data Privacy: Balancing transparency with the need for user data confidentiality.
- Consensus Latency: Minimizing delays caused by blockchain consensus mechanisms.

2: Literature Survey

2.1 Overview of Blockchain-Based DAO

In general case of centralization, the use of a database is basically based on the trust of a third-party organization. For example, as the third party, people all trust the banking system, which can correctly manage the database and record our transactions. The bank keeps accounts for every transaction, and only the bank has the authority to keep users' accounts. However, the shortcoming of such centralized organization is obvious. No one can make sure that whether the centralized organization that manages the database is entirely trustworthy.
For example, during the global economic crisis in 2008, the US government could issue money indefinitely, because it is the central institution of monetary management.

On the contrary, decentralization means that the database
does not depend on a specific organization or administrator but is distributed among all peers. Blockchain is essentially a decentralized database. Each full node has a complete copy of the blockchain ledger. If the database is modified, the information saved by all nodes will be noticed. Thus, the information
in the blockchain database will be open and transparent. Decentralization
solves the trust problem through redundant data validation.

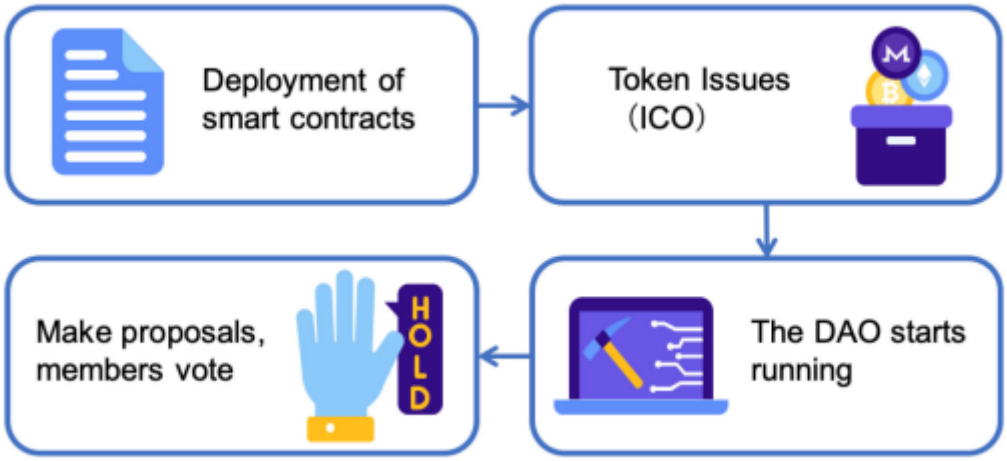
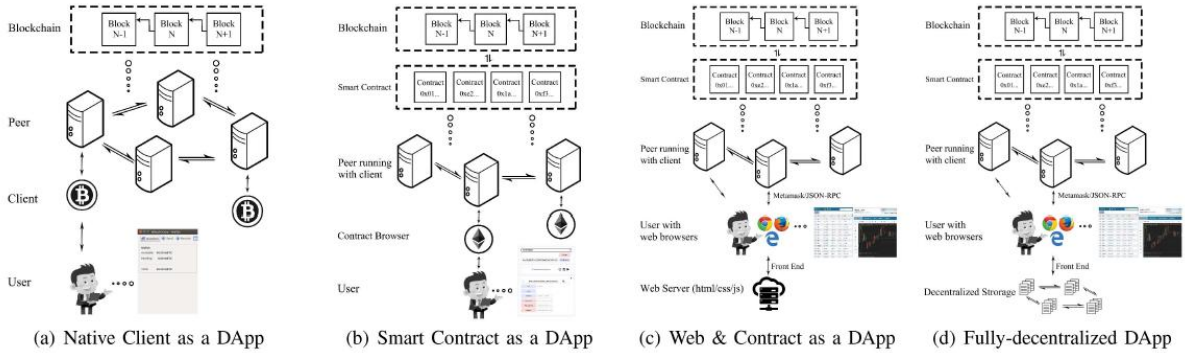


Fig 2.1: Structure of a DApp

The operation of launching a DAO project includes the following steps, which are also illustrated in Fig. 2.1:

1) Developing and deploying smart contracts according to predefined rules.
2) Handling the token issues (through ICO) at the initial financing stage.
3) At the end of the financing phase, a DAO starts running.
4) Proposals are made and members can vote on them.

2.2 Survey on Blockchain-Based Dapp



(a) Native Client as a DApp (b) Smart Contract as a DApp (c) Web & Contract as a DApp (d) Fully-decentralized DApp

Fig 2.2: Architectures of DApp

A decentralized application is an application that does not be controlled by a centralized organization. The motivation for decentralized applications is that traditional centralized application/structure is very vulnerable to attacks and breed corruption. The concept of the decentralized application was proposed much earlier than blockchain. For example, BitTorrent is a decentralized application, and so does much peer-to-peer software. Different architectures of DApps are proposed in the following subsection.

A. Native Client as a DApp

Bitcoin can be considered as one of the blockchain-based decentralized applications in payment. Every user runs a client on a peer and then joins the peer-to-peer network. Because the ledger of payment is decentralised, people can use their own client (e.g., BitcoinWallet) to transfer Bitcoin to other people. Because the user only uses the client to interact with the network, the client is a DApp. In this survey, this architecture is called Native Client as a Dapp. It is used by most of the early Bitcoin-like cryptocurrencies, such as Litecoin, PPcoin, and so on. The shortcoming of this architecture is that the blockchain is customized for the application (e.g., payment).

B. Smart Contract as a Dapp

In the “Native Client as a DApp,” modifying the blockchain for new applications is hard. And the developers of every new DApp need to develop a new blockchain and client, which reduces the efficiency. This can be solved by smart contracts.

Developers can use smart contracts on the blockchain (e.g., Ethereum) to record any information they want. Thus, the DApp developers can choose to write a smart contract as a DApp for the users. Taking Ethereum as an example, if the developers want to develop a DApp for transferring tokens, they can write a token contract within 100 lines of code on Ethereum. Then the users can use a smart contract browser (e.g., Remix, Mist, etc.) to load a contract on the Ethereum network and call the functions written by the developers. In some cases, the client is also the contract browser. This architecture is called Smart Contract as a Dapp.

However, the bottleneck of this architecture is that it requires the users to have some basic knowledge of programming. Meanwhile, the contract browsers of many platforms are not so easy for users because few of them have graphic user interfaces.

C. Web & Contract as a Dapp

To improve the bottlenecks of Contract as a DApp and make it easier for users to use DApp, most DApp developers create a web front end for the smart contracts. The front end is provided as web pages, including the graphic user interface written in code of HTML/CSS/JS. And the web browsers run the JavaScript (or install MetaMask) to connect to the blockchain peers. There are also some light clients that set web browsers and wallets together so that it will be easier for users to use DApps.

Note that the Web client in this architecture is different from the Native client before. The peers can be remote or local. Finally, the browsers can get the important information (e.g., balance, token) from the blockchain and then present it to the frontend. This Web & Contract as a DApp is widely used by most DApps. The main idea is to store the GUI (Graphic User Interface) on the website and the important information (e.g., balance) on the blockchain. This seems to be more familiar to users.

D. Fully-Decentralised DApp

Taylor Gerring proposes an architecture, which can obliterate the notion of separating content from presentation by removing the need to have servers at all. It consists of three modules: Ethereum for decentralized logic, Swarm for decentralized storage, and Whisper for decentralized messaging. This architecture is called Fully-decentralized DApp.

If this concept could be totally implemented, then the developers and users will use the Fully decentralised DApp. The major difference between fully decentralised DApp and Web & Contract as a DApp lies in the storage of the front-end. The storage of Fully-decentralised DApp does not depend on the centralized service but on the decentralised file systems.

2.3 Decentralised Banking Services

Blockchain based crypto based currencies have made it possible to implement a fund transfer system in a more efficient, hassle-free and secure way. The present banking system contains human intervention at many stages wherein system users can modify bank details, bank balance in a fraudulent way. In the classical banking system, users have to wait for a longer time to deposit and withdraw money. A decentralized banking system allows a user to carry out various transactions and manage one's own data securely. Thus, a decentralized banking system has no room for fraudulent activities and causes no capital loss.

3: Technology Stack

3.1: User Interface

The point of interaction between a user and a digital device, application, or website. It encompasses all the visual and interactive elements that allow a user to interact with and control a system. This includes screens, buttons, icons, and other visual elements. The frontend client for the Blockchain Wallet is built using HTML, CSS, JavaScript, and jQuery.

3.1.1: HTML

Hypertext Markup Language is the standard markup language for documents designed to be displayed in a web browser. It defines the content and structure of web content. HTML markup consists of several key components, including those called tags (and their attributes), character-based data types, character references and entity references. HTML tags most commonly come in pairs like <h1> and </h1>, although some represent empty elements and so are unpaired, for example <img>. The first tag in such a pair is the start tag, and the second is the end tag (they are also called opening tags and closing tags).

3.1.2: CSS

Cascading Style Sheets (CSS) is a style sheet language used for specifying the presentation and styling of a document written in a markup language such as HTML or XML (including XML dialects such as SVG, MathML or XHTML). CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

CSS is designed to enable the separation of content and presentation, including layout, colours, and fonts. This separation can improve content accessibility, since the content can be written without concern for its presentation; provide more flexibility and control in the specification of presentation characteristics; enable multiple web pages to share formatting by specifying the relevant CSS in a separate .css file, which reduces complexity and repetition in the structural content; and enable the .css file to be cached to improve the page load speed between the pages that share the file and its formatting.

3.1.3: JS

JavaScript, often abbreviated as JS, is a programming language and core technology of the World Wide Web, alongside HTML and CSS. Ninety-nine percent of websites use JavaScript on the client side for webpage behaviour.

Web browsers have a dedicated JavaScript engine that executes the client code. These engines are also utilized in some servers and a variety of apps. The most popular runtime system for non-browser usage is Node.js.

JavaScript is a high-level, often just-in-time–compiled language that conforms to the ECMAScript standard. It has dynamic typing, prototype-based object-orientation, and first-class functions. It is multi-paradigm, supporting event-driven, functional, and imperative programming styles. It has application programming interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the Document Object Model (DOM).

3.1.4: jQuery

jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers. With a combination of versatility and extensibility, jQuery has changed the way that millions of people write JavaScript.

3.2: Backend

Backend refers to the server-side or infrastructure that powers an application, managing data, logic, and server-side functions. It's the part of an application that users don't directly see but is essential for the application to function correctly. For the backend logic in this Blockchain Wallet, the Solidity programming language is utilised, and for it to work, various dependencies and frameworks are also required, which are also described in detail in this report.

3.2.1: Solidity

Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs that govern the behaviour of accounts within the Ethereum state.

Solidity is a curly-bracket language designed to target the Ethereum Virtual Machine (EVM). It is influenced by C++, Python, and JavaScript. You can find more details about which languages Solidity has been inspired by in the language influences section.

Solidity is statically typed, supports inheritance, libraries, and complex user-defined types, among other features. With Solidity, you can create contracts for uses such as voting, crowdfunding, blind auctions, and multi-signature wallets.

3.3: Dependencies and Frameworks

Package dependencies refer to the external modules, libraries, or packages that a project relies on to function correctly. These dependencies are typically listed in a file like package.json (in Node.js) or setup.py (in Python) and specify which packages and their versions are required for the project.

Frameworks are more than just libraries; they are structured sets of code, conventions, and tools that provide a foundation for building an application. They offer pre-built components, patterns, and structures that developers can use to create applications efficiently. Examples of frameworks include React, Angular, and Spring.

3.3.1: Node Package Manager

npm is the standard package manager for Node.js.

In September 2022 over 2.1 million packages were reported being listed in the npm registry, making it the biggest single language code repository on Earth, and you can be sure there is a package for (almost!) everything.

It started as a way to download and manage dependencies of Node.js packages, but it has since become a tool used also in frontend JavaScript.

npm consists of three distinct components:

- the website
- the Command Line Interface (CLI)
- the registry

Use npm to…

- Adapt packages of code for your apps, or incorporate packages as they are.
- Download standalone tools you can use right away.
- Run packages without downloading using npx.
- Share code with any npm user, anywhere.
- Restrict code to specific developers.
- Create organizations to coordinate package maintenance, coding, and developers.
- Form virtual teams by using organizations.

- Manage multiple versions of code and code dependencies.
- Update applications easily when underlying code is updated.
- Discover multiple ways to solve the same puzzle.
- Find other developers who are working on similar problems and projects.

3.3.2: Truffle Framework

A world class development environment, testing framework and asset pipeline for blockchains using the Ethereum Virtual Machine (EVM), aiming to make life as a developer easier. With Truffle, you get:

- Built-in smart contract compilation, linking, deployment and binary management.
- Advanced debugging with breakpoints, variable analysis, and step functionality.
- Use console.log in your smart contracts
- Deployments and transactions through MetaMask with Truffle Dashboard to protect your mnemonic.
- External script runner that executes scripts within a Truffle environment.
- Interactive console for direct contract communication.
- Automated contract testing for rapid development.
- Scriptable, extensible deployment & migrations framework.
- Network management for deploying to any number of public & private networks.
- Package management with NPM, using the ERC190 standard.
- Configurable build pipeline with support for tight integration.

3.3.3: Ganache

Ganache is a personal blockchain for rapid Ethereum and Filecoin distributed application development. You can use Ganache across the entire development cycle; enabling you to develop, deploy, and test your DApps in a safe and deterministic environment.

Ganache comes in two flavours: a UI and CLI. Ganache UI is a desktop application supporting Ethereum and Filecoin technology. Our more robust command-line tool, ganache, is available for Ethereum development. It offers:

- console.log in Solidity
- Zero-config Mainnet and testnet forking
- Fork any Ethereum network without waiting to sync
- Ethereum JSON-RPC support
- Snapshot/revert state
- Mine blocks instantly, on demand, or at an interval
- Fast-forward time
- Impersonate any account (no private keys required!)
- Listens for JSON-RPC 2.0 requests over HTTP/WebSockets
- Programmatic use in Node.js
- Pending Transactions

3.3.4: MetaMask

MetaMask is many things. For our purposes, it is a gateway to the Blockchain network. The MetaMask extension in a compatible browser such as Google Chrome or Brave or Microsoft Edge allows for web DApps to:

- Interact with users' Ethereum accounts.
- Send transactions and sign messages.
- Listen to account and network changes.

The MetaMask Wallet API is designed for web DApps that connect to the MetaMask browser extension. If you're building a desktop web-only DApp, the Wallet API provides everything you need.

You can connect your DApp to users' MetaMask wallets by detecting MetaMask in their browsers and connecting to their accounts. This approach allows you to detect multiple installed wallets and connect to them without conflicts.

MetaMask injects a global JavaScript API into websites visited by its users using the window.ethereum provider object. This API is specified by EIP-1193, and it allows DApps to request users' EVM accounts, read data from blockchains the user is connected to, suggest that the user sign messages and transactions, and more.

The MetaMask Ethereum provider API contains the following:

- Properties - The provider contains a property that detects if a user has MetaMask installed.
- Methods - The provider contains methods that dapps can call. The request() provider method wraps the JSON-RPC API; dapps can use this provider method to call any RPC method.
- Events - The provider emits events that dapps can listen to.

3.3.5: Mocha Testing Framework

Mocha is a feature-rich JavaScript test framework running on Node.js and in the browser, making asynchronous testing simple and fun. Mocha tests run serially, allowing for flexible and accurate reporting, while mapping uncaught exceptions to the correct test cases.

4: Workflow

4.1: Project Structure

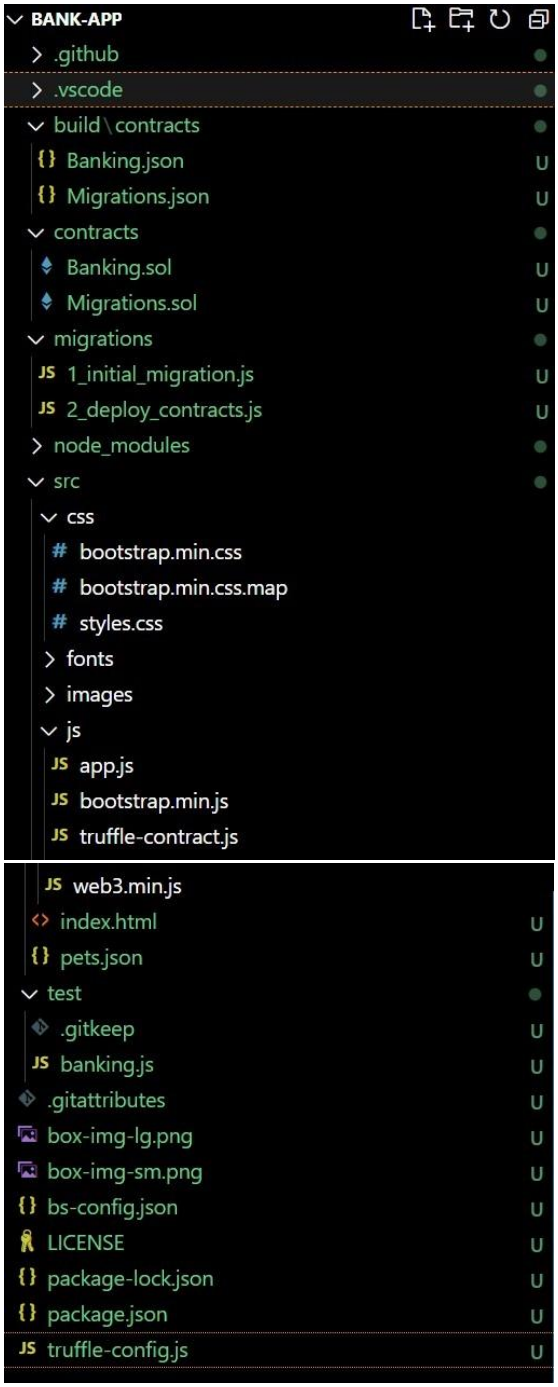


Fig 4.1: Project Structure

4.2: Architecture

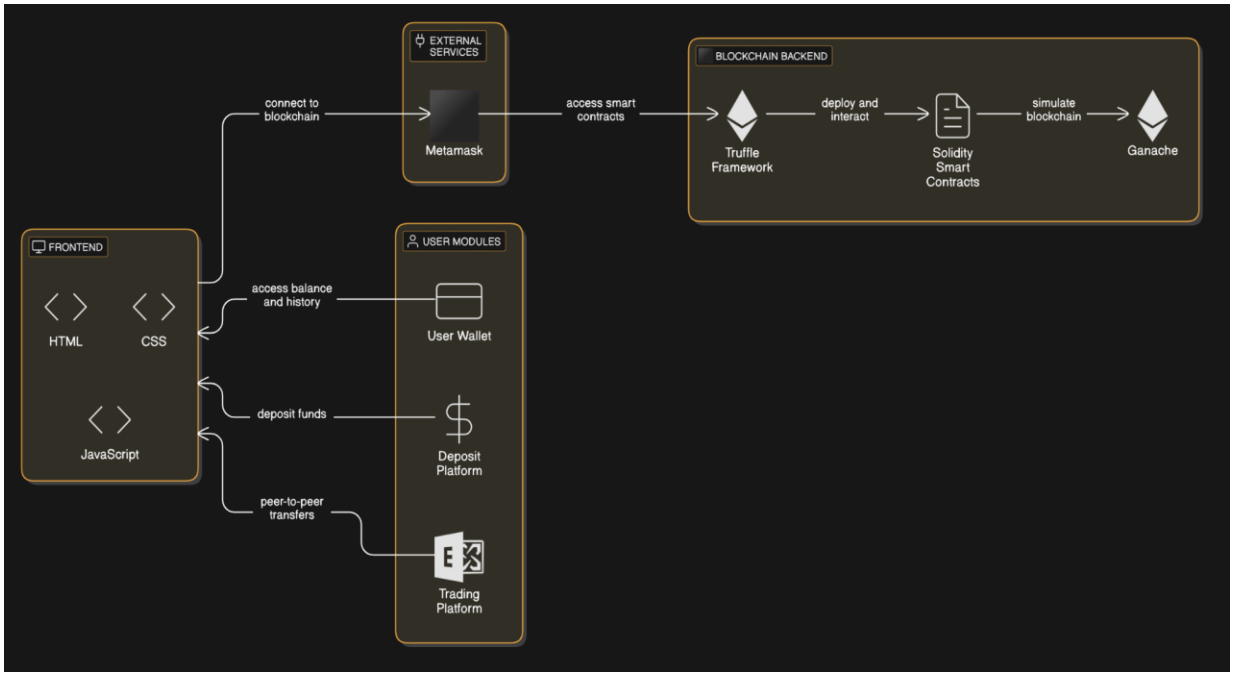


Fig 4.2: Architecture Diagram

The architecture of a blockchain-based decentralized application (DApp) comprises four key players: the frontend, external services, user modules, and the blockchain backend. These components work together to create a secure, user-friendly, and decentralized environment for managing digital assets and transactions.

The frontend is built using standard web technologies such as HTML, CSS, and JavaScript. It provides the user interface and enables interaction with the application. Users can view their wallet balances, transaction history, deposit funds, and initiate peer-to-peer transfers directly from the browser. The frontend also handles communication with external services and backend components, ensuring smooth user experiences across different functionalities.

MetaMask serves as the primary external service, acting as a bridge between the frontend and the blockchain. It is a browser extension that functions as an Ethereum wallet and enables users to sign transactions securely. MetaMask allows the frontend to access blockchain smart contracts and facilitates interactions with deployed contracts without exposing users' private keys or requiring them to leave the application interface.

The user modules represent the core functionalities offered by the DApp. The User Wallet module enables users to monitor their account balances and review transaction history. The Deposit Platform allows users to add funds to their

DApp account. Meanwhile, the Trading Platform provides peer-to-peer transfer capabilities, enabling decentralized exchange of funds or tokens between users. These modules work in conjunction with the frontend and smart contracts to manage user assets effectively.

The blockchain backend consists of the Truffle Framework, Solidity Smart Contracts, and Ganache. Truffle is a development suite that facilitates the compilation, deployment, and testing of smart contracts. Smart contracts are written in Solidity and define the logic governing transactions, fund transfers, and other operations within the DApp. Ganache is used to simulate a blockchain environment locally, allowing developers to test and debug contracts without needing to interact with a live blockchain network.

Overall, this architecture allows users to securely interact with a decentralized application, enabling asset management and peer-to-peer transactions while leveraging the trust less and transparent nature of blockchain technology. The modular and layered design ensures scalability, ease of testing, and integration with blockchain services.

4.3: Flow of the Application

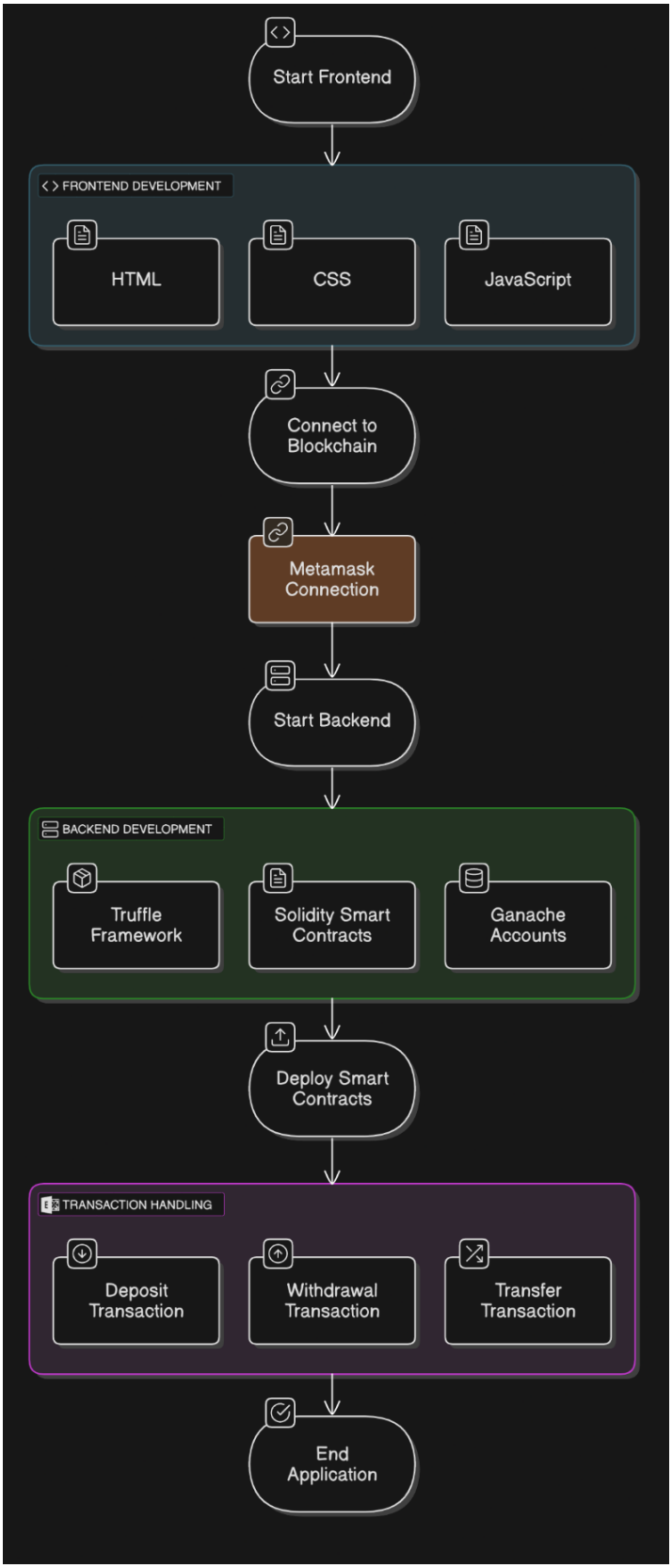


Fig 4.3: Flow Diagram

This flow diagram illustrates the lifecycle of a blockchain-based decentralized application (DApp) from frontend development to backend integration and transaction handling. The process begins with the initiation of the frontend, developed using core web technologies—HTML, CSS, and JavaScript—which together provide the user interface and client-side functionality of the DApp. Once the frontend is ready, the next step is to connect the application to the blockchain. This is achieved through MetaMask, a browser extension that enables secure communication with the blockchain network. MetaMask handles user authentication and transaction signing, bridging the frontend and blockchain layers.

Following a successful connection, the backend development is initialized. This stage involves three key components: the Truffle Framework for compiling and deploying smart contracts, Solidity Smart Contracts that encapsulate the core logic and operations of the DApp, and Ganache Accounts, which simulate a local Ethereum blockchain environment for testing and development purposes. After backend setup, the smart contracts are deployed onto the blockchain (simulated via Ganache in development). Once deployed, the application enters the transaction handling phase, where various user-initiated financial operations are managed. This includes deposit transactions (adding funds), withdrawal transactions (retrieving funds), and transfer transactions (peer-to-peer fund movement).

The process concludes with the termination of the application session, completing the full cycle of user interaction, blockchain connection, smart contract deployment, and transaction execution in the DApp.

4.4: Functionality

The architecture and end-to-end execution flow of a blockchain-based Decentralized Application (DApp), highlighting the integration of frontend technologies, backend development, smart contract deployment, transaction management, and automated testing has so far been described. The architecture is modular, scalable, and designed to support secure peer-to-peer financial interactions through Ethereum smart contracts.

4.4.1: Frontend Development and Initialization

The application lifecycle begins with the frontend development, where foundational web technologies—HTML, CSS, and JavaScript—are used to create an interactive user interface. The frontend facilitates user access to all core features of the DApp, including wallet balance views, deposit forms, transaction history, and peer-to-peer transfer inputs.

Once the UI is complete, the application proceeds to connect to the blockchain through MetaMask, a browser extension wallet. MetaMask plays a crucial role by acting as a key management system for the user's Ethereum accounts. It enables the DApp to:

- Request account access
- Retrieve wallet addresses
- Sign transactions securely
- Track account balances and activities
- Manage Ethereum test accounts

Through MetaMask, the DApp gains read and write access to the blockchain via the user's selected account and network.

4.4.2: Backend Development and Blockchain Simulation

After establishing the MetaMask connection, the backend development process is initiated. This layer involves the use of several Ethereum development tools:

- Truffle Framework: Serves as the project scaffolding and smart contract management tool. It compiles, migrates (deploys), and interacts with smart contracts in a structured environment.

- Solidity Smart Contracts: The core application logic resides in these contracts, which handle deposits, withdrawals, transfers, and balance management. Functions are written to include access control, value validations, and event emission for off-chain tracking.
- Ganache: A local in-memory Ethereum blockchain used for testing and development. Ganache provides a set of deterministic Ethereum accounts preloaded with test ETH, allowing developers to simulate blockchain interactions without incurring real gas costs or relying on testnets.

Developers launch the backend environment using Truffle, and smart contracts are deployed onto the local Ganache blockchain. Contract migration scripts manage the deployment process, ensuring all contracts are compiled and deployed in the correct sequence.

4.4.3: Smart Contract Deployment and Interaction

Once development is complete, the application enters the deployment phase. Using Truffle, the compiled Solidity contracts are deployed to Ganache via migration scripts. These contracts include functionality for:

- Storing and managing user balances
- Validating deposits and withdrawals
- Facilitating peer-to-peer transfers
- Emitting blockchain events (e.g., DepositMade, FundsTransferred, WithdrawalCompleted) for frontend tracking

Upon deployment, each contract is assigned a unique Ethereum address, which the frontend uses (via Web3.js or Ethers.js) to interact with the blockchain.

4.4.4: Automated Testing with Mocha

To ensure contract integrity, automated testing is conducted using Mocha, a JavaScript-based testing framework integrated within the Truffle suite. A total of 8 test cases were written and executed, covering key functional areas:

- Deployment validation
- Account balance initialization
- Successful deposits and event emissions
- Withdrawal operations with balance checks
- Peer-to-peer transfer between test accounts

- Access control verification (e.g., preventing unauthorized withdrawals)
- Edge case handling (e.g., zero-value transfers)
- Reversion of invalid transactions with proper error messages

All 8 test cases passed, confirming that the smart contracts behave as expected under normal and boundary conditions. The tests used the Ganache-supplied test accounts and relied on Web3 calls to simulate real transaction scenarios.

4.4.5: Transaction Handling

With smart contracts successfully deployed and tested, the DApp transitions to the transaction execution phase. Users are able to perform the following operations through the frontend:

- Deposit Transactions: Initiated via MetaMask, the transaction sends ETH to the contract address and updates the sender's balance in the smart contract state.
- Withdrawal Transactions: Users can withdraw available funds back to their MetaMask wallet. The contract validates the balance and processes the transfer.
- Transfer Transactions: Enables peer-to-peer fund transfers between registered accounts. The contract adjusts balances and emits events for tracking.

Each transaction triggers corresponding smart contract functions and requires user approval through MetaMask. After submission, the transaction is mined by Ganache and the result is reflected in the DApp interface.

4.4.6: Completion and Application Shutdown

After transactions are completed, the DApp session can be safely terminated. All blockchain interactions are recorded in the local ledger simulated by Ganache. Since data on Ganache is not persistent between sessions by default, developers may export or snapshot the chain state for further testing or auditing.

5: Client-Side Application

5.1: Local Application

5.1.1: Migrating Smart Contracts

```
D:\Studies\SRP Lab\bank-app>truffle migrate --reset

Compiling your contracts...
===========================
> Compiling .\contracts\Banking.sol
> Compiling .\contracts\Migrations.sol
> Artifacts written to D:\Studies\SRP Lab\bank-app\build\contracts
> Compiled successfully using:
   - solc: 0.5.16+commit.9c3226ce.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)
```

```
1_initial_migration.js
======================

   Replacing 'Migrations'
   ----------------------
   > transaction hash:    0x47063664170cdbce707f30589f9c604aacd59459e076194ad6f8971e26f3439d
   > Blocks: 0            Seconds: 0
   > contract address:    0x0BD5006b99c525E0546ec4799DdC03a3744531F9
   > block number:        37
   > block timestamp:     1746864482
   > account:             0xf541f1289799cD5bBeA269A667aE25EebCC4d468
   > balance:             99.979739489021332748
   > gas used:            193243 (0x2f2db)
   > gas price:           2.509974111 gwei
   > value sent:          0 ETH
   > total cost:          0.000485034927131973 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000485034927131973 ETH
```

```
2_deploy_contracts.js
=====================

   Replacing 'Banking'
   -------------------
   > transaction hash:    0xb59ce2d7615c09629eb012f0b0dd527f91ff490e58ac2ea5c706f5da39759f7e
   > Blocks: 0            Seconds: 0
   > contract address:    0x2a7CbB0148E79B154054b0a1a352F486EF463177
   > block number:        39
   > block timestamp:     1746864482
   > account:             0xf541f1289799cD5bBeA269A667aE25EebCC4d468
   > balance:             99.97795916800703127
   > gas used:            664180 (0xa2274)
   > gas price:           2.50771412 gwei
   > value sent:          0 ETH
   > total cost:          0.0016655735642216 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.0016655735642216 ETH

Summary
=======
> Total deployments:   2
> Final cost:          0.002150608491353573 ETH
```

Fig 5.1: Truffle Contract Migration

5.1.2: Running Tests on Mocha Framework

```
D:\Studies\SRP Lab\bank-app>truffle test
Using network 'development'.


Compiling your contracts...
===========================
> Compiling .\contracts\Banking.sol
> Compiling .\contracts\Migrations.sol
> Artifacts written to C:\Users\TC\AppData\Local\Temp\test--9124-joi0cZ6t25cm
> Compiled successfully using:
   - solc: 0.5.16+commit.9c3226ce.Emscripten.clang


  Contract: Banking
    √ should allow users to deposit ether (52ms)
    √ should allow users to withdraw ether (82ms)
    √ should emit Withdrawal event on successful withdraw (80ms)
    √ should allow users to check their balance (38ms)
    √ should allow users to transfer ether to another user (92ms)
    √ should emit Transfer event on successful transfer (79ms)
    √ should revert withdrawal if insufficient balance (360ms)
    √ should revert transfer if insufficient balance


  8 passing (1s)
```

Fig 5.2: Mocha Testing

5.1.3: Running Application

```
D:\Studies\SRP Lab\bank-app>npm run dev

> pet-shop@1.0.0 dev
> lite-server

** browser-sync config **
{
  injectChanges: false,
  files: [ './**/*.{html,htm,css,js}' ],
  watchOptions: { ignored: 'node_modules' },
  server: {
    baseDir: [ './src', './build/contracts' ],
    middleware: [ [Function (anonymous)], [Function (anonymous)] ]
  }
}
[Browsersync] Access URLs:
 --------------------------------------
       Local: http://localhost:3000
    External: http://192.168.56.1:3000
 --------------------------------------
          UI: http://localhost:3001
 UI External: http://localhost:3001
 --------------------------------------
[Browsersync] Serving files from: ./src
[Browsersync] Serving files from: ./build/contracts
[Browsersync] Watching files...
25.05.10 13:41:57 304 GET /index.html
25.05.10 13:42:00 304 GET /css/styles.css
25.05.10 13:42:00 304 GET /images/yuvaraj.jpg
25.05.10 13:42:00 304 GET /images/manobala.jpg
25.05.10 13:42:00 304 GET /js/web3.min.js
25.05.10 13:42:00 304 GET /js/truffle-contract.js
25.05.10 13:42:00 404 GET /images/jamal.jpg
25.05.10 13:42:00 200 GET /js/app.js
```

Fig 5.3: Running Application

5.1.4: Frontend Client

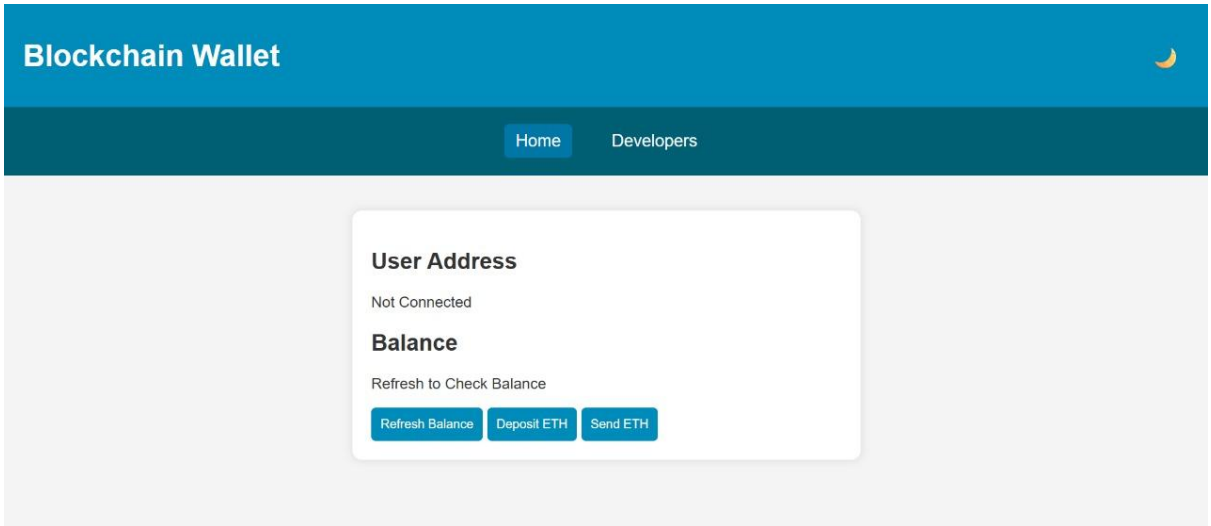


Fig 5.4: Home Page – No User Logged In

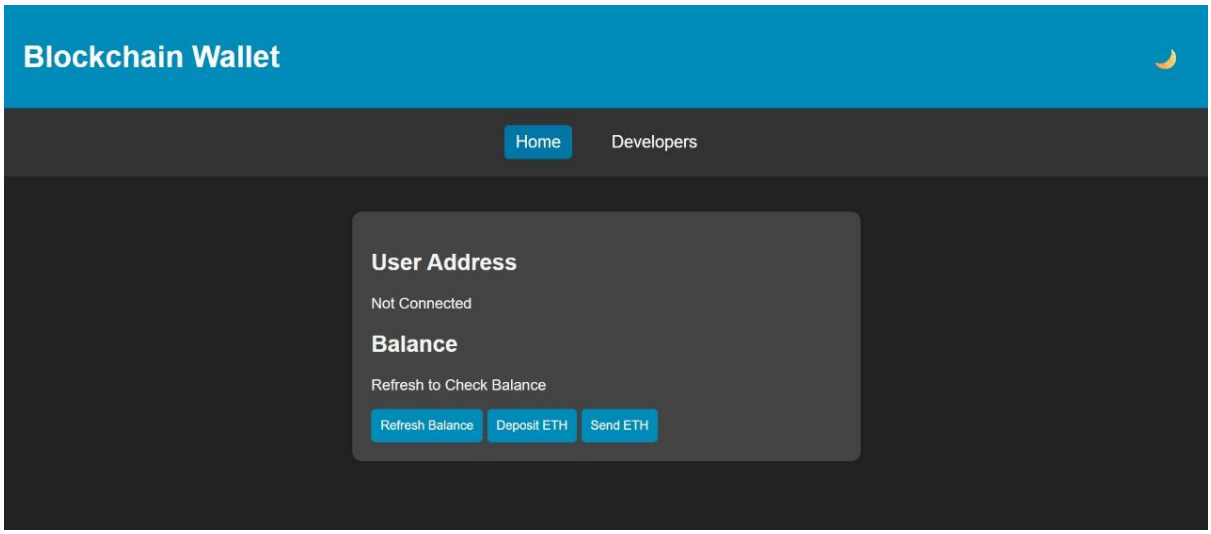


Fig 5.5: Dark Mode

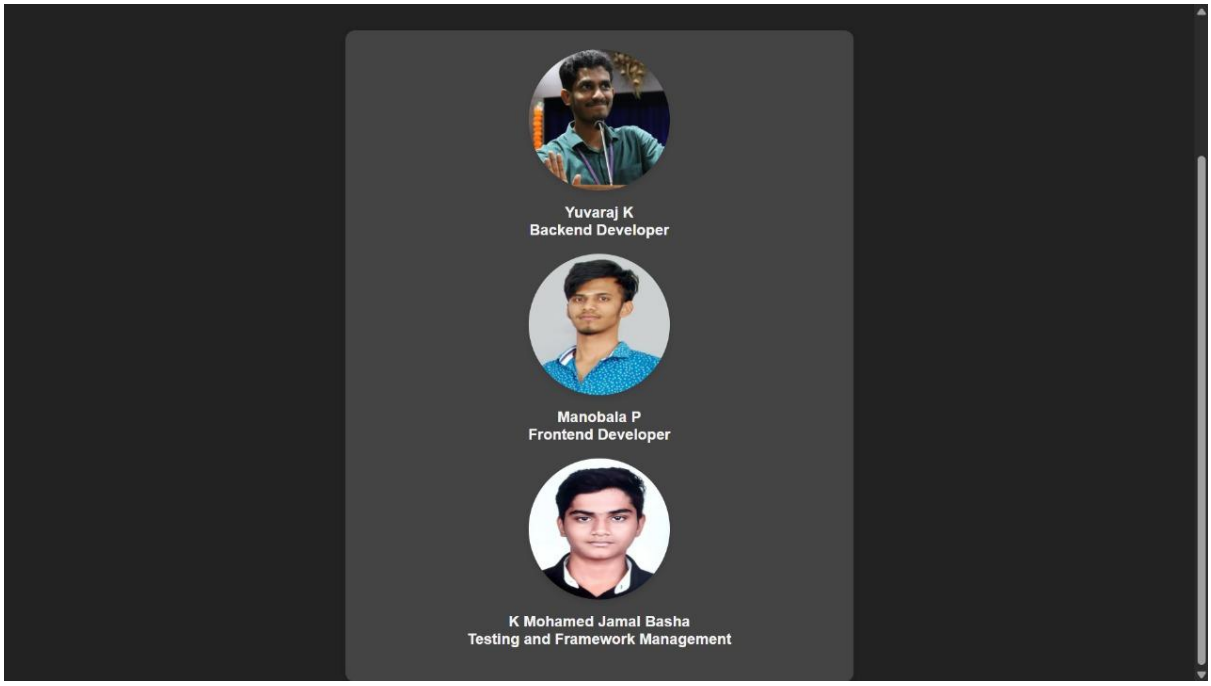


Fig 5.6: Developers Page

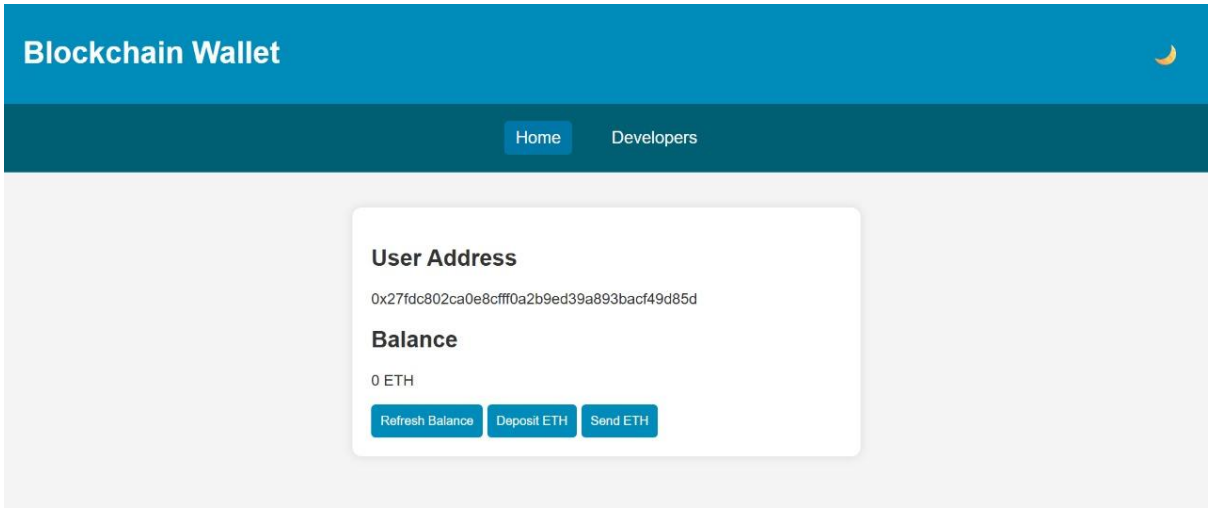


Fig 5.7: After Connecting

Fig 5.8: Deposit

5.2: Ganache Simulation

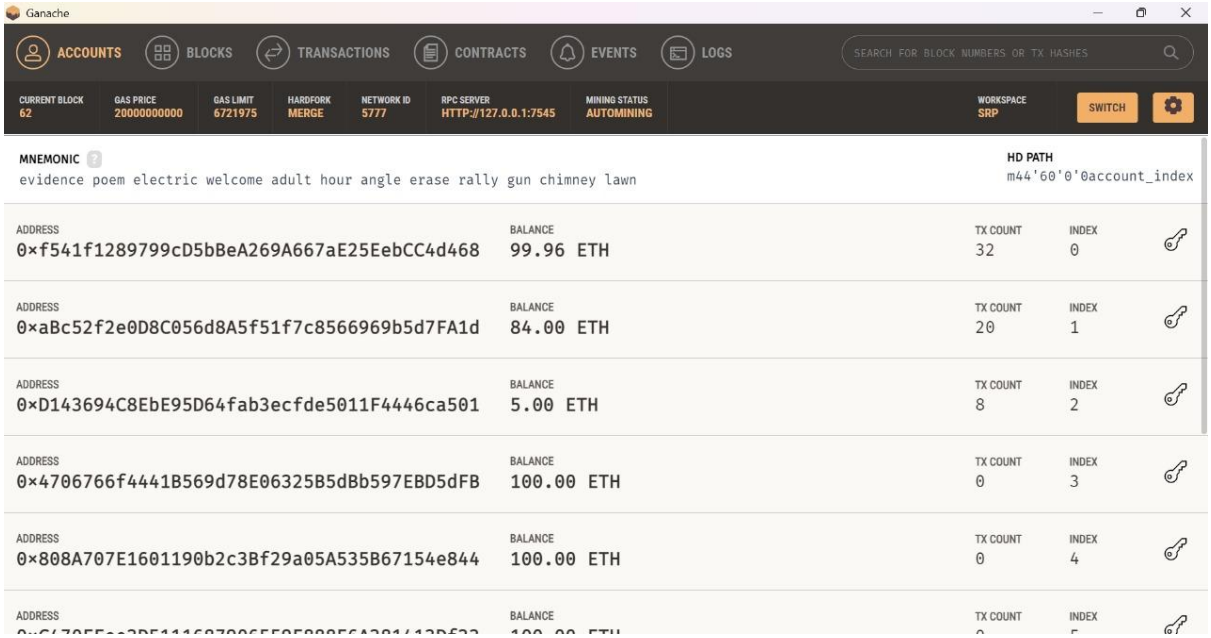


Fig 5.9: Ganache Simulated Accounts

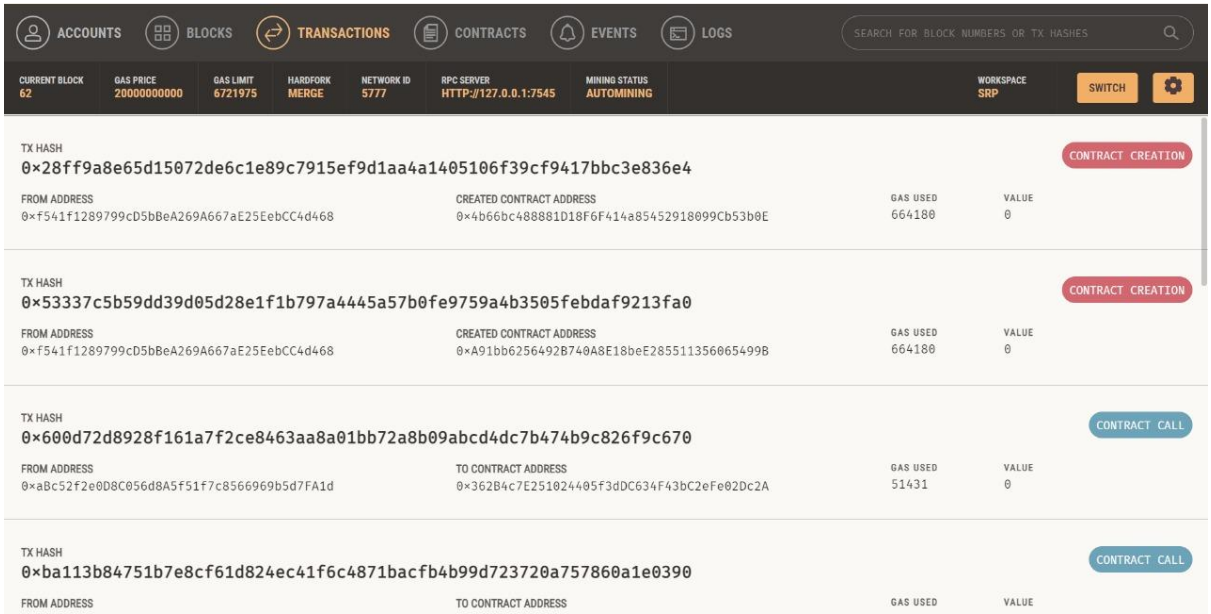


Fig 5.10: Transaction History in Ganache

5.3: MetaMask Connection

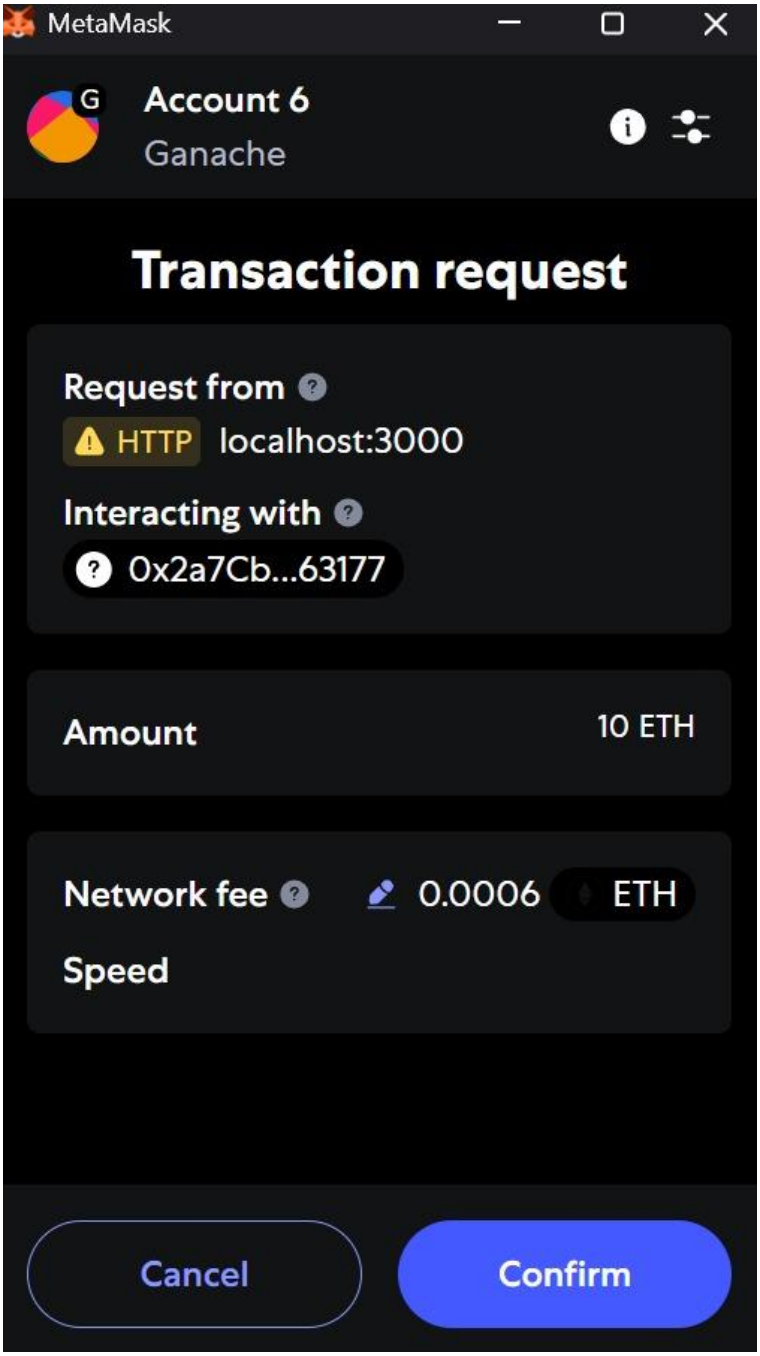


Fig 5.11: Transaction Request

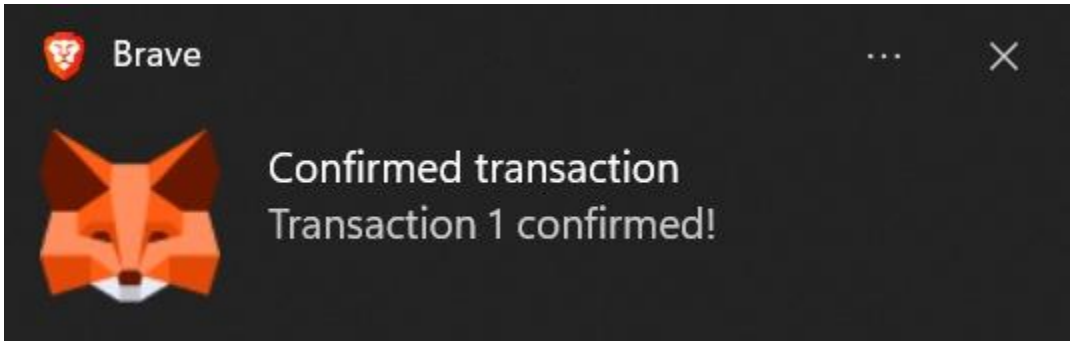


Fig 5.12: Transaction Confirmation

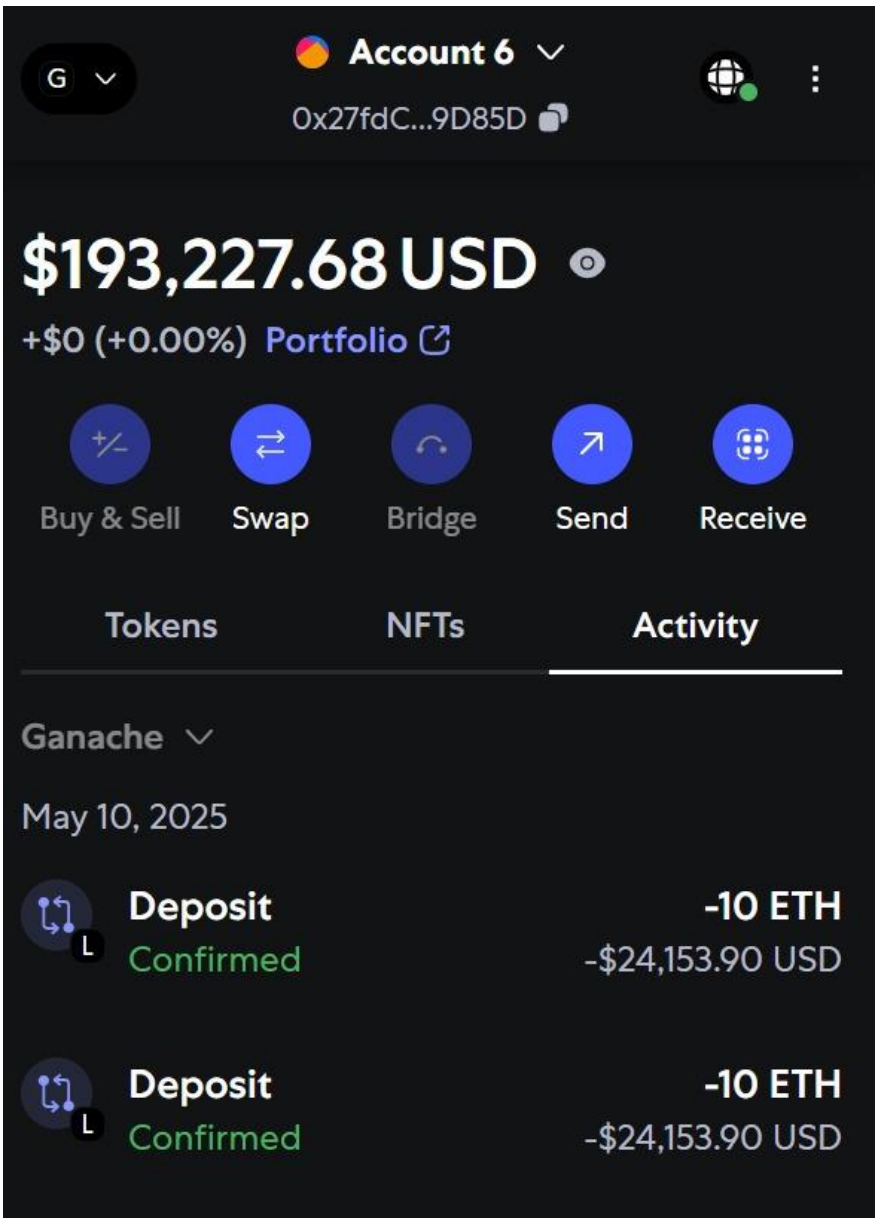


Fig 5.13: Local Transaction History

6: Conclusion

6.1: Potential Improvements

The current architecture of the blockchain-based decentralized banking application demonstrates a solid foundational structure. It uses HTML/CSS/JavaScript for the frontend, integrates MetaMask for blockchain connectivity, and leverages the Truffle framework with Ganache for smart contract deployment and simulation.

While functional, the following enhancements are proposed to improve scalability, security, maintainability, and user experience:

6.1.1. Backend Middleware Integration

Currently, the frontend communicates directly with the blockchain via MetaMask. Introducing a backend API layer can:

- Offload business logic from the frontend.
- Handle error management and logging.
- Provide RESTful endpoints for off-chain operations. This improves the modularity and scalability of the system while enabling easier debugging and user support.

6.1.2. Enhanced Authentication and Identity Management

The system relies solely on MetaMask for user identification. To enhance account security and usability:

- Integrate DID or Web3Auth.
- Implement wallet signature-based session tokens for secure, stateless authentication.

This allows better onboarding, optional KYC compliance, and account recovery mechanisms.

6.1.3. Scalability through Layer-2 Solutions

The architecture currently uses Ganache for local testing, which does not address scalability challenges in live environments. To improve performance:

- Integrate Layer 2 blockchain networks.
- Reduce transaction fees incurred and latency experienced for the end users. This approach supports a higher transaction throughput and improves user experience.

4. Off-Chain Data Management

All interactions are presumed to occur on-chain. However, for optimal performance:

- Use off-chain storage solutions to store non-sensitive metadata, trading history, or analytics.
- Off-chain components can complement on-chain transparency without incurring high gas fees.

5. Modular Smart Contract Design

Currently, smart contracts are monolithic. A modular approach is recommended:

- Split contracts by functionality (e.g., WalletContract, DepositContract, TradeContract).
- Use upgradeable proxy patterns to allow future updates. This enhances maintainability and reduces the scope of security vulnerabilities.

6. Frontend Modernization

The UI is built using basic HTML, CSS, and JS. Adopting modern frameworks improves usability:

- Use React or Vue.js for dynamic components and better state management.
- Integrate Web3 libraries like ethers.js for seamless wallet interaction and contract communication.

7. Smart Contract Testing and Security

While Truffle facilitates smart contract deployment and automated tests using Chai and Mocha, more security testing is critical. It is advisable to:

- Use static analysis tools like Slither, MythX, and OpenZeppelin Defender for security audits.

These practices ensure the robustness and reliability of the deployed smart contracts.

8. Multi-Chain and Interoperability Support

To future-proof the application:

- Consider supporting multiple blockchains (e.g., Ethereum, Binance Smart Chain).
- Implement interoperability standards like ERC-20/721/1155 and cross-chain bridges.

This increases application reach and enhances asset liquidity across ecosystems.

6.2: Conclusion

The proposed blockchain-based decentralized banking application represents a significant step toward redefining traditional financial systems by leveraging the transparency, security, and autonomy offered by blockchain technology. The current architecture provides a functional model that integrates key components such as smart contracts, wallet interfaces, and blockchain simulation environments. However, to transition from a proof-of-concept to a scalable, secure, and user-friendly financial platform, several enhancements are necessary.

Through continuous refinement and adherence to best practices, this decentralized banking system can evolve into a robust, secure, and inclusive financial solution for users worldwide.

7: References

1) L. Liu, S. Zhou, H. Huang and Z. Zheng, "From Technology to Society: An Overview of Blockchain-Based DAO," in IEEE Open Journal of the Computer Society, vol. 2, pp. 204-215, 2021, doi: 10.1109/OJCS.2021.3072661.
keywords: {Blockchains;Security;Organizations;Bitcoin;Databases;Fault tolerant systems;Computer hacking;Blockchain;contracts;DAO;fault tolerant system;governance}

2) P. Zheng, Z. Jiang, J. Wu and Z. Zheng, "Blockchain-Based Decentralized Application: A Survey," in IEEE Open Journal of the Computer Society, vol. 4, pp. 121-133, 2023, doi: 10.1109/OJCS.2023.3251854.
keywords: {Decentralized applications;Blockchains;Smart contracts;Economics;Consensus protocol;Service-oriented architecture;Security;Blockchain;decentralized application}

3) H. Bhat, G. Bank, Y. Jawale, R. Wairkar and S. Mirchandani, "Decentralized Banking Services using Blockchain Technology," 2023 8th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 2023, pp. 515-521, doi: 10.1109/ICCES57224.2023.10192758.
keywords: {Costs;Economic indicators;Employment;Banking;Real-time systems;Blockchains;Peer-to-peer computing;Blockchain;Crypto Based Currency;Banking System;Decentralized;Lending;Trading}

4) HTML: https://developer.mozilla.org/en-US/docs/Web/HTML
5) CSS: https://developer.mozilla.org/en-US/docs/Web/CSS
6) JS: https://developer.mozilla.org/en-US/docs/Web/JavaScript
7) jQuery: https://jquery.com/
8) DApp Tutorial: https://www.dappuniversity.com/articles/the-ultimate-ethereum-dapp-tutorial
9) npm: https://nodejs.org/en/learn/getting-started/an-introduction-to-the-npm-package-manager
10) npm: https://docs.npmjs.com/
11) Solidity: https://docs.soliditylang.org/en/v0.8.30/
12) Truffle: https://archive.trufflesuite.com/docs/truffle/
13) Ganache: https://archive.trufflesuite.com/docs/ganache/
14) MetaMask: https://docs.metamask.io/wallet/
15) Mocha: https://mochajs.org/